

Manuel d'Installation et de Configuration

Ce document s'adresse à l'équipe technique chargée du déploiement de l'application **RailMind Lite** sur les serveurs de CAMRAIL.

1. Prérequis Système

Infrastructure matérielle (Recommandée)

- **CPU** : 4 Cœurs
- **RAM** : 8 Go minimum (pour l'exécution locale de modèles et le traitement des embeddings).
- **Stockage** : 50 Go (SSD recommandé pour la base de données PostgreSQL).

Dépendances Logicielles

- **Système d'exploitation** : Linux (Ubuntu 22.04 LTS) ou Windows Server.
- **Base de données** : PostgreSQL (Version 15+) avec l'extension `pgvector` installée.
- **Backend** : Python 3.10+
- **Frontend** : Node.js 18+

2. Étape 1 : Installation de la Base de Données

RailMind Lite nécessite PostgreSQL et son extension vectorielle pour le RAG.

1. Installer PostgreSQL.
2. Installer l'extension `pgvector` :

```
sudo apt install postgresql-15-pgvector
```

3. Créer la base de données :

```
CREATE DATABASE camrail_rda;  
\c camrail_rda  
CREATE EXTENSION vector;
```

3. Étape 2 : Configuration du Backend (API)

Le backend gère l'intelligence artificielle, l'authentification et l'indexation.

1. **Cloner le code source** : Placez-vous dans le répertoire du serveur et récupérez le dossier `backend`.

2. **Créer un environnement virtuel Python** :

```
python3 -m venv venv
source venv/bin/activate
```

3. **Installer les dépendances** :

```
pip install -r requirements.txt
```

4. **Configuration (`.env`)** : Créez un fichier `.env` à la racine du backend. Voici un exemple complet :

```
# Base de données
DATABASE_URL=postgresql://utilisateur:motdepasse@localhost:5432/camrail_rda

# Sécurité
SECRET_KEY=votre_cle_secrete_generee_aleatoirement

# Configuration LLM (Google Gemini Lite)
LLM_PROVIDER=gemini
GEMINI_API_KEY=votre_cle_api_google_ai

# Single Sign-On (Microsoft Active Directory) - Remplir avec les identifiants IT
MICROSOFT_CLIENT_ID=
MICROSOFT_CLIENT_SECRET=
MICROSOFT_TENANT_ID=common
```

5. **Initialiser la base de données** : Exécuter les migrations automatiques pour créer les tables :

```
alembic upgrade head
```

6. **Données de démonstration (Seed)** : Pour générer l'administrateur par défaut et peupler la base avec les groupes de sécurité :

```
python seed_demo.py
```

7. Lancement de l'API :

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

4. Étape 3 : Configuration du Frontend (Interface Utilisateur)

1. Naviguer vers le dossier `camrail-rda` .
2. **Installer les dépendances Node.js :**

```
npm install
```

3. **Configuration de l'environnement :** Vérifier ou créer un fichier `.env.local` contenant l'URL de l'API Backend :

```
NEXT_PUBLIC_API_URL=http://votre-serveur-ip:8000/api/v1
```

4. Compiler et Lancer :

```
npm run build  
npm run start
```

5. Tests Post-Installation

1. Accédez à l'interface via `http://votre-serveur-ip:3000` .
2. Connectez-vous avec le compte `admin@camrail.net` (mot de passe par défaut : `camrail2026`).
3. Téléversez un document de test.
4. Allez dans l'onglet Assistant et posez une question relative au document pour valider la chaîne de recherche (Embeddings + LLM).

6. Surveillance et Maintenance

- **Logs :** Les journaux du backend (`uvicorn`) afficheront les erreurs de traitement.
- **Sauvegarde :** Planifier un dump quotidien de la base `camrail_rda` (via `pg_dump`).

- **Gestion des doublons :** Le système bloque automatiquement les uploads de fichiers ayant un même hash. En cas d'erreur de checksum, vérifiez l'intégrité du fichier.